Atelier 1

Introduction : qu'est-ce qu'une édition structuée ? + XML : principes et syntaxe

---

Sommaire

1 Qu'est-ce qu'un texte ? 2

2 On distingue trois *états* du texte 2

3 Les formats textuels 2

4 Qu'est-ce qu'une édition critique ? 3

4.1 Peut-on (sérieusement) utiliser un traitement de texte pour l'édition scientifique ? . . 3

4.2 Un document structuré n'est pas un document mis en page 4

4.3 Implémentation 5

4.4 Pourquoi un format plein texte ? 5

5 Caractéristiques du XML 5

6 Standard 6

7 Historique 6

8 Principes de conception 6

9 Principes syntaxiques 7

10 DES langages XML 7

11 Premier contact avec un document XML 7

12 Passage à l'édition publiée en HTML : confronter la source et le résultat 8

13 De quoi est composé un document XML ? 8

14 Le prologue 9

15 Les éléments 9

16 Les attributs 9

17 Règles d'écriture des identificateurs génériques et des attributs 9

18 Les entités 10

19 Les commentaires **11**

20 Un encodage bien formé : les règles à retenir **11**

21 OHCO **11**

22 *What is Text, Really ?* Le texte serait une hiérarchie ordonnée d'objets de contenu **11**
22.1 Autres modèles 11
22.2 Contournements en TEI 12

1 Qu'est-ce qu'un texte ?

- Le texte linguistique
- La matérialité (un support, un format, un aspect… tout cela peut intéresser les chercheurs et chercheuses)
- Des métadonnées sur le texte lui-même, sur les conditions de publication ou sur la source d'une édition
- La structure…

Le sens d'un texte ne repose pas seulement sur son contenu linguistique, mais aussi sur sa structure : un livre se divise en chapitres annoncés par des titres et une table des matières, en paragraphes, et s'accompagne de notes de bas de page ou de références bibliographiques, autant de repères qui permettent au lecteur de discerner où commencent et s'achèvent les enchaînements de la pensée, ce qui serait bien plus difficile face à un flux ininterrompu de mots. Même les espaces entre les mots, invention relativement récente dans l'histoire de l'écriture, servent d'indices visuels, permettant de discrétiser les éléments du texte : l'unité de base est le mot plutôt que la lettre. Cette organisation prend d'ailleurs des formes variées selon les genres : chants, strophes et vers pour un poème ; actes, scènes, répliques et didascalies pour une pièce de théâtre ; entrées, définitions, exemples et étymologies pour un dictionnaire. La structure fait donc partie intégrante du sens. Par exemple, le nom "Hamlet", revêt un sens différent selon qu'il est prononcé par un personnage ou qu'il figure dans la liste des personnages en tête d'une scène.

2 On distingue trois *états* du texte

- texte brut : le contenu textuel, dépourvu de toute structure et mise en forme
- texte balisé : le contenu textuel structuré à l'aide de balises
- texte formaté : le texte mis en forme à l'aide d'un éditeur de texte

3 Les formats textuels

Il y a trois grandes catégories de formats textuels qui recouvrent les trois états du texte.

Tout d'abord, on a le texte brut, comme un fichier `.txt`. C'est la forme la plus simple : le fichier contient essentiellement une suite de caractères, sans information particulière sur la structure ou la présentation du document. Son principal avantage est donc son interopérabilité : pratiquement

n'importe quel système peut le lire. En revanche, cette simplicité implique un manque d'information : on ne sait pas, par exemple, qu'un passage correspond à un titre, à une liste ou à un paragraphe.

Ensuite, on trouve le texte balisé, avec des formats comme `Markdown`, `LaTeX`, `XML` ou `HTML`. Ici, on ajoute des informations qui permettent de décrire la structure du contenu. Par exemple, on peut indiquer qu'un élément est un titre, une liste ou une section. On introduit donc une forme de sémantisme, tout en conservant l' interopérabilité parce que ce sont des formats plein texte. Mais c'est difficilement lisible par un humain.

Enfin, on a le texte formaté, avec des formats comme `ODT`, `DOCX` ou `Pages`, qui reposent en réalité sur le format XML, mais qui va loin dans la présentation. C'est le principe du WYSIWYG. On utilise une interface graphique qui recrée la manière dont le texte va s'afficher ou être imprimé. Mais on mélange la forme et la structure sémantique. Et le fichier devient également plus complexe et dépend davantage du logiciel qui permet de le créer ou de l'interpréter.

Il y a une sorte de progression du texte, de son état le plus brut vers son état balisé OU formaté. On ajoute de l'information sur la structure ET/OU sur la présentation. Le formatage introduit une grande complexité et de nombreuses dépendances.

4 Qu'est-ce qu'une édition critique ?

- Sous ce terme, on regroupe plusieurs types d'éditions qui poursuivent des objectifs différents. L'édition critique propose un texte reconstitué à partir de témoins multiples, manuscrits, copies ou éditions successives, parfois contradictoires. L'édition diplomatique transcrit le plus fidèlement possible un témoin unique, avec ses abréviations, ses graphies et ses particularités. L'édition génétique s'intéresse enfin au processus d'écriture : elle restitue les brouillons, les ratures et les strates de corrections successives.

Ces éditions n'ont pas le même but, mais elles sont toutes un essai de reconstitution ou de représentation d'un texte, et non le texte lui-même. Elles partagent donc les mêmes exigences.

- D'abord, l'objectivité, ou en tout cas la transparence. L'éditeur fait des choix : il corrige une leçon, développe une abréviation, décide comment représenter une rature. Le lecteur doit pouvoir comprendre pourquoi on lui présente telle version, pourquoi on représente tel aspect de la page mais pas un autre, donc telle représentation du texte plutôt que telle autre.
- Enfin, une édition critique doit être fiable et consistante. Fiable, parce qu'on doit pouvoir s'appuyer sur elle pour travailler. Consistante, parce que les mêmes situations doivent être traitées de la même façon d'un bout à l'autre : si une correction est signalée d'une certaine manière à la page 10, elle doit l'être de la même manière à la page 200.

4.1 Peut-on (sérieusement) utiliser un traitement de texte pour l'édition scientifique ?

Avec ces exigences en tête :

- Premier problème : la distinction entre forme et contenu. Dans un traitement de texte, ce que vous voyez à l'écran est mélangé à ce que vous voulez dire. Un mot en italique, est-ce un titre d'œuvre, une citation, un mot étranger, une emphase ? Le traitement de texte ne le sait pas : il ne connaît que l'apparence, l'italique.
- Deuxième problème : la documentation de l'édition. Une édition critique doit expliquer ses choix : quels manuscrits ont été utilisés, quelles sont les conventions adoptées, qui a fait quoi. Cette documentation peut inclure des informations sur l'édition en train de se faire, ou sur les sources et leur état matériel. La TEI offre un cadre pour renseigner cela, que le traitement de texte oublie.
- Troisième problème : le contrôle de la cohérence des choix éditoriaux. Dans un traitement de texte, rien ne vous empêche de faire des incohérences : un titre peut mis en forme à la main de deux façons différentes par exemple. Avec XML TEI, il existe des mécanismes de validation : on peut vérifier automatiquement que l'édition respecte les règles qu'on s'est données.
- Quatrième problème : l'obsolescence et l'interopérabilité. Un fichier issu de traitement de texte impose une quasi-équivalence entre le format et un logiciel souvent propriétaire. Et qu'en est-il de la transformation vers d'autres formats ?

Ce n'est pas le bon format pour réaliser une édition numérique et garantir l'interopérabilité des données textuelles. Il appauvrit les possibilités de représentation du texte (en tant qu'objet multiple) et limite les possibilités d'exploitation. Ces limites sont d'ailleurs souvent contournées à l'aide de transformations complexes vers des formats plus sémantiques.

4.2 Un document structuré n'est pas un document mis en page

Une édition scientifique permet d'interroger le texte au-delà de son aspect typographique.

Le point de départ de tout projet d'édition, c'est l'analyse qui consiste à percevoir une hiérarchie dans un document, en fonction de ce à quoi on s'intéresse dans ce document. Un même document peut se prêter à plusieurs analyses hiérarchiques.

Par exemple un ouvrage en prose peut être envisagé comme une succession de chapitres composés de paragraphes OU comme un ensemble de pages composées de lignes. Ces différentes hiérarchies sont souvent indépendantes les unes des autres et sont difficilement superposables. Ainsi un paragraphe peut chevaucher deux pages. Donc souvent, il faut choisir de privilégier l'une ou l'autre des hiérarchies. Et cela constitue déjà une interprétation, une perspective sur notre objet d'étude. C'est déjà une représentation. On se concentre uniquement sur les éléments pertinents au regard de nos besoins spécifiques.

Structurer un document, c'est choisir quelles informations sont pertinentes.

4.3 Implémentation

Grâce au balisage on insère l'information dans le document de manière à ce qu'elle soit lisible par les machines. Et le grand avantage de choisir le modèle arborescent de XML, c'est que les ordinateurs peuvent traiter facilement ces hiérarchies ordonnées, bien plus vite et plus efficacement que les textes non hiérarchisés.

Représenter sous forme d'arborescence = être capable de gérer et de manipuler de grandes quantités de données.

Le choix de ce modèle du texte sert aussi de protocole. Cela permet aussi de rester consistant lors de l'édition.

4.4 Pourquoi un format plein texte ?

- Maintenabilité : un fichier texte reste lisible, peu importe le système, on peut l'ouvrir, l'éditer avec n'importe quel éditeur de texte.
- Portabilité : ces formats sont indépendants d'un logiciel ou d'une plateforme particulière. Un fichier texte peut être transféré d'un système à un autre et rester lisible.

Il est important de séparer autant que possible le contenu de l'outil qui permet de le manipuler ou de le présenter. Ce principe facilite la conservation du document dans le temps, ainsi que les échange entre différents environnements.

5 Caractéristiques du XML

a) Un métalangage extensible

XML ne définit pas un vocabulaire de balises fixe (contrairement à HTML) : il fournit des règles permettant de créer ses propres langages de balisage adaptés à un domaine donné (XHTML, TEI, MathML, SVG, etc.). D'où son nom : *Extensible Markup Language*.

b) Structure arborescente et hiérarchique

Un document XML est constitué d'éléments imbriqués formant une arborescence stricte : chaque élément ouvrant doit être fermé, et les éléments ne peuvent se chevaucher (un document doit être *bien formé*).

c) Balisage descriptif

Un balisage qui décrit la fonction d'un élément plutôt que son apparence : par exemple `<titre>` plutôt que des instructions de mise en gras ou de taille de police.

d) Neutralité vis-à-vis du domaine d'application

XML n'est pas conçu pour un usage particulier : il sert aussi bien à l'encodage de textes savants

(TEI), à l'échange de données entre systèmes informatiques, qu'à la configuration de logiciels ou à l'encodage des journaux de ces mêmes logiciels.

6 Standard

XML est un sous-ensemble de SGML, entièrement décrit par sa spécification, et par construction, tout document XML est un document SGML conforme. Il est maintenu par le W3C (*World Wide Web Consortium*), l'organisme de standardisation du Web, ce qui en garantit :

- Une spécification publique et stable, librement consultable
- Une gouvernance ouverte, avec un processus de recommandation impliquant les membres du consortium
- Une interopérabilité recherchée

En cela il doit faciliter :

- L'échange de données
- La migration vers d'autres plateformes, d'autres logiciels, d'autres formats

Le rôle du W3C dans l'établissement d'une recommandation est d'attirer l'attention sur la spécification et de promouvoir son adoption large, ce qui renforce la fonctionnalité et l'interopérabilité du Web — un rôle que XML a rempli au-delà des espérances initiales, donnant naissance à toute une famille de technologies dérivées (XSLT, XML Schema, XPath, XQuery, SVG, RSS/Atom, DocBook, et bien sûr des applications comme la TEI).

7 Historique

Le XML s'inscrit dans une longue généalogie du balisage de documents :

- 1967-1973 : premiers travaux sur le balisage générique ("generic markup") = il faut décrire la structure logique d'un document avant tout, pas son apparence visuelle. Ces travaux aboutissent au *Generalized Markup Language* (GML), développé chez IBM.
- 1986 : le Standard *Generalized Markup Language* (SGML) devient une norme ISO (ISO 8879). C'est un métalangage réputé complexe, ce qui freine son adoption sur le Web naissant. Adopté par la TEI en 1987.
- En parallèle développement et adoption du HTML comme standard du web.
- 1996 : le W3C lance des travaux visant à rendre SGML utilisable sur le Web, sous la houlette d'un groupe de travail (*XML Working Group*) présidé par Jon Bosak. 1998 : XML 1.0 devient une recommandation officielle du W3C.

8 Principes de conception

- XML ne sert pas à afficher les données, mais à les décrire. Seul, il ne fait rien.
- Le nom des balises n'est pas prédéfini : on peut librement créer son propre vocabulaire.

- On peut utiliser une “grammaire” de balises (un schéma), pour définir des contraintes ou s’assurer de la consistance de l’encodage.
- XML est auto-descriptif et lisible par l’homme. Un fichier XML n’est rien que du texte, on peut facilement prendre connaissance d’un corpus dans n’importe quel éditeur textuel.

9 Principes syntaxiques

- Toute donnée textuelle est entourée de balises : une balise ouvrante et une balise fermante, qui délimite le contenu et lui attribue une signification.
- Les éléments sont inclus les uns dans les autres, emboîtés comme des poupées russes, dans une logique strictement hiérarchique.
- Le principe d’héritage découle de la structure arborescente. un élément enfant hérite du contexte défini par ses éléments parents, ce qui signifie que sa position dans l’arbre porte en elle-même une information sémantique : un même mot balisé `<nom>` n’aura pas la même portée selon qu’il se trouve à l’intérieur d’un élément `<auteur>` ou d’un élément `<personnage>`.

10 DES langages XML

On dit des **applications** XML.

XML est un métalangage qui sert de fondation à de nombreuses applications spécialisées, chacune définissant son propre vocabulaire de balises adapté à un domaine particulier.

- Dans le champ des humanités numériques et des sciences de l’information, la `TEI` (*Text Encoding Initiative*) définit des balises pour l’encodage de textes littéraires et savants (structure, variantes, annotations critiques).
- `EAD` (*Encoded Archival Description*) propose un vocabulaire dédié à la description normalisée des fonds d’archives (inventaires, cotes, notices).
- `SVG` (*Scalable Vector Graphics*) utilise XML pour décrire des images vectorielles interprétables par les navigateurs web, et `MathML` fait de même pour la notation mathématique.
- On trouve également `XHTML`, une reformulation d’`HTML` respectant les règles strictes du `XML`.
- `RSS` et `Atom`, employés pour la syndication de contenus web (flux d’actualités, podcasts).
- D’autres applications servent des besoins plus transversaux, comme `XML Schema` ou `RelaxNG`, qui définissent eux-mêmes des règles de validation pour d’autres documents `XML`.

11 Premier contact avec un document XML

Ceci est l’édition électronique intégrale des notes tenues en 1437 par Pierre Christofle, notaire royal attaché à la prévôté d’Orléans. Pierre Christofle fut notaire royal à Orléans de 1423 à 1450, attaché à la prévôté pour qui il rédigeait des contrats portant le sceau de l’institution, et comme ses confrères, il inscrivait dans un registre et en une forme abrégée les conventions qu’il attestait.

387 notes sont rédigées entre le 1er janvier et le 29 décembre 1437, elles sont conservées aux Archives

départementales du Loiret. Nous avons devant les yeux une édition numérique de sources notariales médiévales, encodée en XML-TEI, qui permet l'enquête sur le lexique des actes et dans la version HTML, elle propose une vue en parallèle des images numérisées du registre et de leur transcription.

- ✻ Repérez la première ligne : à votre avis à quoi sert-elle ?
- ✻ Repérez le texte balisé.
- ✻ Sur un court passage, dessinez l'arborescence.
- ✻ Repérez la transition entre les métadonnées et le contenu propre.

12 Passage à l'édition publiée en HTML : confronter la source et le résultat

- ✻ Prenez la minute 119 du 2 avril. La même personne est désignée par quatre graphies différentes : *Jean Charbonnier, Jean, Jehan Charbonnier, Charbonnier*. Comment cela est-il visible dans la page web ?
- ✻ Observez l'index "Types d'actes" ou "Lieux et personnes" : à partir de quelles balises ce type d'index a-t-il pu être généré automatiquement ?

`<persName xml:id="minute-119-p-001" ref="#p-0261">Étienne le Clerc</persName>` encode un nom de personnage.

- ✻ Prenez la minute 1 du 1er janvier : Un même mot du manuscrit peut apparaître différemment que sur le manuscrit : par exemple "qui vaille et puisse valoir a touz jours ladicte rente de vingt sols parisis et mieulx et paier avec ce toutes tailles d'eglise, de ville, de pu[is]". Quelle balise TEI permettent de coder à la fois la forme originale et sa résolution ?

`pu<supplied>is</supplied>` code une graphie restituée.

13 De quoi est composé un document XML ?

- ✻ **Le prologue**
- ✻ **Les éléments** : Les balises ainsi que leur contenu
- ✻ **Les attributs** : Les "caractéristiques" d'une balise
- ✻ **Les entités** : Appel à un caractère ou une séquence prédéfinie
- ✻ **Les commentaires** : Pour extraire des passages du XML

```
<?xml version="1.0" encoding="UTF-8"?>
<root>
 <element><!-- comment --></element>
 <empty/>
 <el attribute="value">&amp;amp;</el>
```

```
</root>
```

14 Le prologue

Il indique la version de la recommandation XML qui sert de base à l'écriture du fichier (il n'y a eu à cette date qu'une seule version, la version 1.0), ainsi que l'encodage de caractères utilisé ⟶ `<?xml version="1.0" encoding="UTF-8"?>`

15 Les éléments

XML modèle la hiérarchie d'un document en utilisant les éléments, qui consistent en :

- une balise ouvrante
- une balise fermante
- un contenu = tout ce qui se passe à l'intérieur des balises

L'élément n'est pas la balise : l'élément c'est la balise et tout ce qu'il y a au milieu, qui peut être du texte et/ou des commentaires, et/ou d'autres éléments.

L'élément c'est pas non plus juste le nom de l'élément, souvent appelé *generic identifier*.

Attention les éléments doivent être strictement imbriqués les uns dans les autres.

16 Les attributs

Les attributs sont du balisage supplémentaire qui donne des informations en plus sur l'élément. Par exemple, si vous voulez baliser le texte en langue étrangère : soit vous utilisez trois balises différentes `<english>` `<german>` `<chinese>`, soit vous utilisez une seule balise `<foreign>` en ajoutant l'information sur la langue dans l'attribut `lang='fr'`.

Les attributs sont écrit à l'intérieur de la balise ouvrante, et ils sont écrit selon la structure `nom de l'attribut = "valeur de l'attribut"`.

17 Règles d'écriture des identificateurs génériques et des attributs

La chaîne de caractères formant le nom **peut** contenir :

- les caractères alphanumériques (lettres de a à z, et de A à Z, chiffres de 0 à 9, caractères non latins)
- le tiret bas ou *underscore* _
- le trait d'union -
- et le point .

La chaîne de caractère formant le nom **ne peut pas** :

- contenir tout autre signe de ponctuation
- contenir des espaces

- ✻ commencer par un nombre, ni par un point, ni par un trait d'union
- ✻ commencer par xml (quelle que soit la casse utilisée)

Attention, XML est sensible à la casse !

18 Les entités

Si XML utilise les chevrons pour déclarer son balisage, comment faire pour représenter le chevron en tant que caractère, en faisant qu'il soit échappé, et pas interprété ? C'est un des caractères dits **réservés**, c'est-à-dire qui ne peut pas être représenté directement dans le texte. Le deuxième caractère réservé emblématique est l'esperluette, justement parce qu'elle permet de déclarer des entités. Ces deux caractères doivent être remplacés dans le document balisé par des entités. Les entités commencent par une esperluette et se terminent par un point-virgule, la partie entre les deux indique la signification.

Concernant les caractères spéciaux non réservés, cela peut être une bonne pratique de les insérer comme des entités, mais elles peuvent aussi être saisies sous forme de texte brut.

On utilise donc les entités pour insérer :

- ✻ les carcatères réservés
- ✻ les caractères invisibles ou difficiles à identifier (espace insécable, espace cadratin, espace sans chasse, trait d'union insécable)
- ✻ les caractères Unicode rares ou spécialisés, difficile à retrouver sur le clavier ou à identifier dans le document (le glyphe peut ne pas être pris en charge par la font)

Dans les deux derniers cas, il faut utiliser les références numériques des caractères. L'esperluette initiale est alors suivie du caractère dièse.

Exemple :

` ` ⟶ Le signe dièse indique qu'il s'agit d'une référence numérique de caractère et non d'une entité de caractère. La lettre x signale que le nombre est exprimé en hexadécimal (exprimé en base 16). Un nombre décimal (base dix) aurait pu être utilisé à la place : ` ` signifie la même chose. La signification de ces nombres (autrement dit, les caractères qu'ils représentent) repose sur la norme Unicode.

Exemple tout bête : je veux écrire la lettre é. L'ordinateur ne stocke pas é. Il stocke les nombres qui représentent les caractères, et pour que tous les systèmes puissent se comprendre, on se repose sur le standard unicode qui fait une liste de tous les caractères dont on peut avoir besoin (du moins, on espère) et les numérote.

Je veux le caractère Unicode qui porte le numéro `00E9`, j'écris donc `é`.

Cela permet de représenter un caractère sans avoir à écrire directement ce caractère dans le fichier.

Et comme c'est un peu lourd d'écrire ça tout le temps, on peut utiliser le principe de remplacement : on déclare l'entité dans une DTD, puis dans l'encodage on utilise plutôt un nom mnémotechnique.

```
<!ENTITY eaigu "&#xe9;">
```

Et dans mon encodage, j'écris : `&eaigu;`

Le principe de remplacement peut être aussi utilisé pour des séquences de caractères qui reviennent souvent. Par exemple si je sais que je vais répéter tout le temps Text Encoding Initiative tout au long du texte, je peux déclarer :

```
<!ENTITY TEI "Text Encoding Initiative">
```

Et dans mon encodage j'écris : `&TEI;`

19 Les commentaires

Servent à… commenter, utiles pour documenter ce qu'on fait ou consigner des notes. Lors d'un traitement informatique, ils sont ignorés par défaut.

20 Un encodage bien formé : les règles à retenir

- Balise ouvrante = balise fermante
- 1 seul élément racine
- Respect de l'imbrication
- 1 attribut du même nom par balise

21 OHCO

22 *What is Text, Really?* Le texte serait une hiérarchie ordonnée d'objets de contenu

Le XML est essentiellement une hiérarchie d'éléments, et les balises ne sont qu'un moyen de représenter cette hiérarchie de manière linéaire (séquentielle). C'est pour ça que lorsqu'on ouvre un élément, il faut le fermer ; et lorsqu'on l'ouvre dans un contexte particulier (par exemple, un autre élément), il est nécessaire de le fermer avant de pouvoir quitter ce contexte.

Derose et al. dans un article important qui date de 1993 et qui s'appelle *What is texte really ?* (rien que ça) explique que tout texte peut être représenté selon ce modèle où les éléments s'emboîtent les uns dans les autres selon une hiérarchie stricte. Avec des parents, des enfants et dans les enfants des frères et sœurs qui s'enchaînent. C'est le modèle OHCO, *Ordered Hierarchy of Content Objects*, en anglais.

22.1 Autres modèles

Tout document constitue-t-il réellement une seule hiérarchie ? Si seulement c'était aussi simple !

Beaucoup de document présentent en réalité des hiérarchies multiples et simultanées présentant un intérêt pour les chercheurs.

Le format XML gère moins efficacement les hiérarchies multiples, simultanées et chevauchantes.

exemple

Un paragraphe (structure logique) commence à la fin d'un folio et se poursuit sur le suivant (structure matérielle) :

```
<!-- NON BIEN FORMÉ : les éléments se chevauchent -->
<folio n="33v">
   <p>Le XXVI jour dudit mois, je encontré Guillaume
</folio>
<folio n="34r">
   de Nades qui me dit la mort de monsegneur le roy Loys.</p>
</folio>
```

Le <p> s'ouvre dans le premier <folio> et se ferme dans le second : c'est invalide. On ne peut pas dire à la fois "ce folio contient ce texte" et "ce paragraphe contient ce texte".

Autre exemple classique : vers et phrases

```
<!-- NON BIEN FORMÉ -->
<l>Le temps s'en va, <s>madame, le temps s'en va</l>
<l>et nous ne sommes plus. </s></l>
```

Le vers (l) et la phrase (s) se chevauchent : la phrase commence au milieu du premier vers et finit dans le second.

22.2 Contournements en TEI

a) Les jalons (milestones) : on privilégie une hiérarchie (ici le paragraphe) et on marque l'autre par un élément vide :

```
<p>Le XXVI jour dudit mois, je encontré Guillaume
   <pb n="34r"/>
   de Nades qui me dit la mort de monsegneur le roy Loys.</p>
```

b) fragmentation : on découpe l'élément en morceaux qui respectent l'arbre, reliés par des attributs :

```
<p xml:id="p1" part="I" next="#p2">Le XXVI jour dudit mois, je encontré Guillaume</p>
<pb n="34r"/>
<p xml:id="p2" part="F" prev="#p1">de Nades qui me dit la mort de monsegneur le roy Loys.</p>
```

C'est le choix le plus courant. Pour les vers, on utiliserait <lb/> ou <milestone/>.

Mais la facilité avec laquelle les ordinateurs traitent les arborescences XML incite les chercheurs à ne pas renoncer à ces avantages pratiques.

Par conséquent, la plupart des projets d'humanités numériques continuent aujourd'hui de s'appuyer sur le XML, en recourant à diverses stratégies pour pallier le problème des hiérarchies chevauchantes.